MATLAB-exercise: Correlation functions and power spectral density for stochastic signals

0. Aim and sequence of the course

The aim of the course is to achieve a fundamental understanding and a physical/intuitive understanding of the concepts autocorrelation function, cross correlation function and power spectral density for discrete time stochastic signals.
These concepts are the foundation for all types of statistically orientated signal processing, e.g. spectral analysis and adaptive systems.
In this exercise focus will primarily be on the basic definition and interpretation of the concepts, where after the following exercises will be studying various applications of partly the correlation functions in themselves and partly in relation to the two above mentioned main areas of application.
The exercise is not focusing directly on the underlying mathematical foundation of the ergodic and stationary stochastic processes of the correlation functions. In the last point of the exercise however, there will be an opportunity to study this more closely.

1. The autocorrelation function, $r_{xx}(k)$

MATLAB's function for calculation of correlation functions is called xcorr
help xcorr
Notice in particular the opportunity of getting the "lag"-values returned.

Example 1: signal = gauss-distributed white noise
```
N = 1024;
x1 = randn(1,N);
n = 0:N-1;
stem(n, x1, ,.')
```

In order to not viewing the same "random" section of signals, a "seed" can be given to the random-generator.
help randn
randn('state',sum(100*clock)) gives a new signal section every time

The autocorrelation function is calculated with the thereto pertaining "lags"
```
[rxx k] = xcorr(x1);
stem(k, rxx, '.')
```

Interpretation, comments, OK?

As to the following examples it would be advantageous to create a small function which plots the signals and the autocorrelation function, save as m-file with the same name.

```
function autokorrplot(x,option);
subplot(2,1,1)
n=0:length(x)-1;
stem(n,x,'.')
subplot(2,1,2)
[rxx k]=xcorr(x,option);
stem(k,rxx,'.')
```

Test the function
autokorrplot(x1,'none')

Example 2: Low pass filtered white noise
fs = 8000;
help fir1
h = fir1 (50, 1000/(fs/2)); filter coefficients for FIR-LPF
freqz(h,1,1024,fs)
x2 = filter(h,1,x1);
autokorrplot(x2,'none')
Comments? Difference in the two plots in relation to the broadband white noise?

Try changing the cut-off frequency of the filter to a lower value, filtrate, plot, observe and comment. As expected?

Example 3: Sinus-signal
f1 = 400;
Ts = 1/fs;
x3 = sin(2*pi*f1*n*Ts);
autokorrplot(x3,'none')
Comments?
Use zoom-function in the plot-window.

Example 4: Sinus-signal + white noise
x4 = x3 + randn(1,N);
autokorrplot(x4,'none')
Use the zoom again.
Comments?

Try changing the signal relationship by means of increasing the noise effect:
sigma = 2; (or even larger)
x4 = x3 + sigma*randn(1,N);
autokorrplot(x4,'none')
Conclusion regarding the characteristics of the cancelling out by averaging feature of the autocorrelation function?

Example 5: white noise with DC-value
DC = 1;
x5 = x1 + DC;
autokorrplot(x5,'none')

Characteristics of the autocorrelation function:

Study again the 5 examples above regarding the following characteristics for $r_{xx}(k)$

$$r_{xx}(0) = \sum x^2(n) = total\ signal - energy$$

$$r_{xx}(-k) = r_{xx}(k) \qquad symmetry, why$$

$$r_{xx}(k) \le r_{xx}(0) \qquad \max in 0,\ why?$$

$$r_{xx}(\infty) = DC - value \qquad why?$$

Estimation and rating of the autocorrelation function:

The mathematical definition of the autocorrelation function is, as you know

$$r_{xx}(k) = E\left[x(n)x(n+k)\right]$$

In praxis – both in MATLAB and in real-time DSP – only a section of signal is present, i.e. the N-samples

x(0), x(1), ... x(N-1).

The most direct way of estimating the theoretical expression is

$$r_{xx}(k) = \sum_{n=0}^{N-1-|k|} x(n)x(n+k) \qquad for\ -(N-1) \le k \le (N-1)$$

This corresponds to the calculated autocorrelation function in the examples above, corresponding to that the "options" in the xcorr-command have been set to "none".

This sum grows according to the length of the signal section, so normally you would want to norm this out in one of more ways.

Study the other options

help xcorr

One method is to set r_{xx}, so that the maximum value is 1

$$r_{xx}(k) = \frac{1}{r_{xx}(0)} \sum_{n=0}^{N-1-|k|} x(n)x(n+k) \qquad for\ -(N-1) \le k \le (N-1)$$

This corresponds to options = 'coeff'

Another way is to set $r_{xx}(k)$ with the number of x-samples that actually is part of the calculation of the k-te value in r_{xx}

$$r_{xx}(k) = \frac{1}{N-|k|} \sum_{n=0}^{N-1-|k|} x(n)x(n+k) \qquad for\ -(N-1) \le k \le (N-1)$$

This corresponds to options = 'unbiased'.

The name indicates that this is the most "true" estimator because the average value of the estimate by means of this value the true value will be $r_{xx}(k)$.

On the other hand, the variant (insecurity) will large at large $|k|$-values, because there is not that many x-samples in the calculation of these.

Finally the options corresponds = 'biased' to the estimate

$$r_{xx}(k) = \frac{1}{N} \sum_{n=0}^{N-1-|k|} x(n)x(n+k) \qquad for\ -(N-1) \le k \le (N-1)$$

This estimate does, as the name implies, not provide the true average value (however asymptotic for $N \to \infty$), but it is nevertheless the most applied due a smaller variance than "unbiased".

Carry out the 5 examples above again, but now experimenting with these estimation-methods, e.g.

autokorrplot(x1,'none')
autokorrplot(x1,'coeff')
autokorrplot(x1,'unbiased')
autokorrplot(x1,'biased')
and correspondingly for x2, x3, x4, x5
Comment on the look of the correlation function, regarding the estimation method.
A general rule is that at these estimation methods the $r_{xx}(k)$-values are only trustworthy for $|k|/N < 10\text{-}20\ \%$.

Connection between autocorrelation and convolution:
Convolution and correlation are closely related operations, as convolution is "flip-and-slide", while correlation is "slide-but-no-flip", or to put it more specifically

$$r_{xx}(k) = x(k) * x(-k)$$

This can be shown by means of a small example, apply for instance " from the above
subplot(2,1,1) ; park plot
[rxx k]=xcorr(x2,'none'); calculation in the usual way
stem(k,rxx,'.')
subplot(2,1,2)
help fliplr
x2mirror = fliplr(x2);
help conv
rxx2 = conv(x2,x2mirror); calculation with convolution
stem(k,rxx2,'.')
Concordance?

Calculation of autocorrelation with own algorithm, corresponding to real-time implementation:
Application of MATLAB's xcorr-command hides the $O(N^2)$-calculation complexity when real-time calculating the autocorrelation function. In order to illustrate this, a selfdefined algorithm can be constructed

```
function [rxx,k] = egen_xcorr (x);
N = length(x);
k = -(N-1):(N-1);
rxx = zeros(1,2*N-1);
for ki=0:N-1
   for n=1:(N-ki)
      rxx(N+ki)=rxx(N+ki)+x(n+ki)*x(n);
   end
   rxx(N-ki)=rxx(N+ki);
end
rxx = rxx/N;
```

Test the function, for instance with

```
subplot(2,1,1)
[rxx k] = xcorr(x2,'biased');
stem(k, rxx, '.')
subplot(2,1,2)
[rxx_egen k_egen] = egen_xcorr(x2);
stem(k_egen, rxx_egen, '.')
```

OK?
The auto-covariance function:
If you want to study stochastic signals with an average value different from 0, the it would be advantageous to use the auto-covariance function
help xcov
Normally, the stochastic signals are anticipated to be without DC-content and Xcov is therefore not at commonly applied as the xcorr.

2. The cross correlation function, $r_{xy}(k)$

While the autocorrelation function compares a signal with a time lagged version of itself, then the cross correlation function compares 2 various mutual delayed signals ***help xcorr***

For use in the following examples, a small function which plots the 2 signals and their cross correlation is created

```
function krydskorrplot(x,y,option);
subplot(3,1,1)
nx=0:length(x)-1;
stem(nx,x,'.')
subplot(3,1,2)
ny=0:length(y)-1;
stem(ny,y,'.')
subplot(3,1,3)
[rxy k]=xcorr(x,y,option);
stem(k,rxy,'.')
```

Example 1: x1 = gauss-distributed white noise, y1 = various variants of x1

```
N = 1024;
x1 = randn(1,N);
y1a = x1;
krydskorrplot(x1,y1a,'none')
```

Kommentarer ?

y1 is set to a time delayed version of x1

```
delay = 300;
y1b = [randn(1,delay)  x1(1:N-delay)];
krydskorrplot(x1,y1b,'none')
```

OK?

is set to a low pass filtered version of x1

```
h = fir1(50, 1000/(fs/2));
y1c = filter(h,1,x1);
krydskorrplot(x1,y1c,'none')
```

Use zoom in the plot, is maximum in k=0 and why not?
Try varying the filter order, e.g. 100, 200 and cut-off frequency, e.g. 500, 200
As expected??

y1 is set to a time delayed and low pass filtered version of x1

```
y1d = filter(h,1,y1b);
krydskorrplot(x1,y1d,'none')
```

OK?

Example 2: x2 = sinus-signal, y2 = various variants of x2

```
f2 = 40;
n = 0:N-1;
x2 = sin(2*pi*f2*n*Ts);
y2a = x2;
krydskorrplot(x2,y2a,'none')
```

y2 is set to a time delayed version of x2
delay = 80;
y2b = [zeros(1,delay) x2(1:N-delay)];
krydskorrplot(x2, y2b, 'none')
OK?
Try setting the delay to 280 and 480 respectively and calculate/plot. Explanation?

y2 is set to a noise infected version of x2
sigma = 1;
y2c = x2 + sigma*randn(1,N);
krydskorrplot(x2,y2c,'none')
What can be concluded regarding the characteristics of the cross correlation function?
Try varying sigma, e.g. to 2, 4, 6, 8, calculate and plot. Comments?

Characteristics of the cross correlation function:
Among the characteristics of the cross correlation function it can – among other things– be said:

$$r_{xy}(k) = r_{yx}(-k) \qquad \textit{while normally} \qquad r_{xy}(-k) \neq r_{xy}(k) \qquad (\textit{why}?)$$

$$\left(r_{xy}(k)\right)^2 \leq r_{xx}(0) r_{yy}(0)$$

$$r_{xy}(k) \leq \frac{1}{2}\left(r_{xx}(0) + r_{yy}(0)\right)$$

Check, in the two examples above, in each case the first of these characteristics by means of changing the order of x and y, e.g.
krydskorrplot(x1,y1b,'none')
krydskorrplot(y1b,x1,'none')
Examine, if necessary, also the other two characteristics (not that important in praxis).

Fixing and estimation methods:
The mathematical definition of the cross correlation function is, as you know:

$$r_{xy}(k) = E\left[x(n+k)y(n)\right]$$

The same methods can be applied for fixing/estimation as studied in relation to the autocorrelation function.
Experiment, for instance, with
krydskorrplot(x1, y1d, 'none')
krydskorrplot(x1, y1d, 'coeff')
Is this result as you expected? Why not?
krydskorrplot(x1, y1d, 'biased')
krydskorrplot(x1, y1d, 'unbiased')
Comments?

Connection between cross correlation and convolution:
Also the cross correlation is closely related to convolution, as

$$r_{xy}(k) = x(k) * y(-k)$$

This can also be demonstrated with a small example, apply – for instance – x2 from above
subplot(2,1,1)
[rxy k] = xcorr(x2,y2b,'none');
stem(k, rxy, '.')
subplot(2,1,2)
rxy = conv(x2,fliplr(y2b));
stem(k,rxy,'.')
OK?

Real-time algorithm for calculation of cross correlation:
If time allows it, then make a $O(N^2)$ direct calculation of rxy, corresponding to previously for the autocorrelation function (not necessary).

3. Power Spectral Density, PSD, $P_{xx}(f)$

The power spectral density, Power Spectral Density PSD, or shortly the power spectra, indicates how the aggregated effect of the stochastic signal is distributed in frequencies, i.e. information about the frequency spectra of the signal.
PSD is mathematically defined by

$$P_{xx}(\omega) = Fourier\{r_{xx}(k)\}$$

For discrete-time signals we are talking about the continuous and periodic DTFT, discrete-time F.T., which can be approached by the sampled version, the discrete and periodic DFT, which can be calculated efficiently with FFT.

In principle, 3 commandoes can be applied in MATLAB

1) freqz(r) calculates DTFT
2) fft(r) calculates DFT
3) psd(r) calculates averaged and over-lapping fft-spectras, more about this in later exercises.

Below here, primarily freqz(r) is used, but when having particularly long signals it is more advantageous to use fft(r) due to calculation time.

Regarding freqz the small detail is that freqz(b,a) in principle calculates the frequency characteristics for a *system* with H(z)-coefficients indicated in the vectors b (numerator) and a (denominator):

$$H(\omega) = \frac{\sum_n b_n \cdot e^{-j\cdot\omega\cdot n}}{\sum_n a_n \cdot e^{-j\cdot\omega\cdot n}}$$

It is, however, seen that if the following is set
b = {r(k)} (sample-values of the time-function) and
a = 1
then the command *freqz (b, a)* will calculate DTFT{r(k)}.

Example 1: signal = gauss-distributed white noise

```
N = 1024;
n = 0:N-1;
x1 = randn(1,N);
[rxx k] = xcorr(x1,'none');
stem(k,rxx,'.')
[Pxx  f]=freqz(rxx,1,1024,fs);
plot(f,10*log10(abs(Pxx)))
```

Comments? As expected?

The power spectra P_{xx} is real (not complex), as it only indicates the power distribution, not phase information.
Examine if this is true:

```
Pxx(1:10)
```

Explanation?
Help: plot the frequency spectra again

```
freqz(rxx,1,1024,fs)
```

Look at the phase and give an explanation!

The reason is that $r_{xx}(k)$ with xcorr is calculated in k-values in the interval [(-(N-1), (N-1)]. These 2N-1 values are stored in the indexes from 1 to 2N-1 due to the array-indexing of MATLAB.
When freqz(r) is applied, the first value in the r-vector is understood as r(0), while it really is r(-(N-1)).
In this way a "false delay" is imported on N-1 samples, hence the phase characteristics above.

In order to get the correct, real PSD calculated, a phase correction corresponding to N-1 samples must be introduced, i.e. a factor

$$\exp\left(+ j * \omega * (N-1)\right) = \exp\left(+ j * \frac{2\pi}{fs} * f * (N-1)\right)$$

subplot(4,1,1)	park figure
plot(f,10*log10(abs(Pxx)))	
subplot(4,1,2)	
plot(f,unwrap(angle(Pxx)))	without phase correction
Pxx = Pxx.*exp(i*(2*pi/fs)*f*(N-1));	phase correction
subplot(4,1,3)	
plot(f,10*log10(abs(Pxx)))	
subplot(4,1,4)	
plot(f,unwrap(angle(Pxx))); axis([0 fs/2 –1 1])	with phase correction

Comments?

Regarding the following examples, the previous autocorrelation plot is modified into being depicting also the PSD

```
function rxxPxxplot(x,option,fs);
subplot(3,1,1)
N=length(x);
n=0:N-1;
stem(n,x,'.')
subplot(3,1,2)
[rxx k]=xcorr(x,option);
stem(k,rxx,'.')
subplot(3,1,3)
[Pxx f]=freqz(rxx,1,1024,fs);
Pxx=Pxx.*exp(i*(2*pi/fs)*f*(N-1));
plot(f,10*log10(abs(Pxx)))
```

Test the function
rxxPxxplot(x1,'none',fs)
Rating of rxx gives correct 0dB-level:
rxxPxxplot(x1,'biased',fs)

Example 2: Low pass filtered white noise

```
h = fir1(50, 1000/(fs/2));
x2 = filter(h,1,x1);
rxxPxxplot(x2,'biased',fs)
```

Comments?
Try changing the cut-off frequency of the filter.

Eample 3: Sinus-signal

```
f1 = 800;
n = 0:N-1;
Ts = 1/fs;
x3 = sin(2*pi*f1*n*Ts);
rxxPxxplot(x3,'biased',fs)
```

Example 4: Sinus-signal + white noise

sigma = 1;
x4 = x3 + sigma*randn(1,N);
rxxPxxplot(x4,'biased',fs) use zoom-function in plots

Comments?
Try increasing sigma, to – for instance - 2, 4, 10

Rating of the autocorrelation function and influence on PSD

Test the various "options" and observe/explain the effect on PSD

rxxPxxplot(x1,'none',fs)
rxxPxxplot(x1,'coeff',fs)
rxxPxxplot(x1,'unbiased',fs)
rxxPxxplot(x1,'biased',fs)

Calculation of PSD in two ways (the Wiener-Kinchine sentence)

As previously examined, the connection is

$$r_{xx}(k) = x(k) * x(-k)$$

In the frequency domain this corresponds to

$$P_{xx}(\omega) = X(\omega) \cdot X^*(\omega) = \left|X(\omega)\right|^2$$

Demonstation by means of a small example:

```
subplot(2,1,1)
[rxx k] = xcorr(x3,'none');
[Pxx f] = freqz(rxx,1,1024,fs);
plot(f,10*log10(abs(Pxx)))
subplot(2,1,2)
[X f] = freqz(x3,1,1024,fs);
Pxx = X.*conj(X);
plot(f,10*log10(abs(Pxx)))
```

What is the fundamental difference between freqz(x) and freqz(r_{xx}) calculated for a stochastic signal?

4. Cross power spectra, $P_{xy}(f)$

The cross power spectral density, or shortly called the cross spectra, provides information about to what extend two different stochastic signals contain the same frequencies.
If, in this way, x(n) contains the frequency f_1 with amplitude A_1 and phase θ_1 and y(n) same frequency f_1 with amplitude A_2 and phase θ_2, then the cross spectra will contain a contribution at f_1 with amplitude $A_1A_2/2$ and phase θ_1-θ_2.
Note, that contrary to PSD, the cross spectrum contains phase-information and is therefore normally complex.
The cross spectrum is mathematically defined by

$$P_{xy}(\omega) = Fourier\left\{r_{xy}(k)\right\}$$

Modify the previous cross correlation plot function into also containing amplitude and phase-characteristics for the cross spectrum, including phase correction due to the array-indexing of MATLAB

```
function rxyPxyplot(x,y,option,fs);
subplot(5,1,1)
N=length(x);
n=0:N-1;
stem(n,x,'.')
subplot(5,1,2)
stem(n,y,'.')
subplot(5,1,3)
[rxy k]=xcorr(x,y,option);
stem(k,rxy,'.')
subplot(5,1,4)
[Pxy f]=freqz(rxy,1,1024,fs);
Pxy=Pxy.*exp(i*(2*pi/fs)*f*(N-1));
plot(f,10*log10(abs(Pxy)))
subplot(5,1,5)
plot(f,unwrap(angle(Pxy)))
```

Example 1: x1 = gauss-distributed white noise, y1 = various variants of x1

```
x1 = randn(1,N);
```

y1 is set to a time delayed version of x1

```
delay = 300;
y1b = [randn(1,delay) x1(1:N-delay)];
rxyPxyplot(x1,y1b,'none',fs)
```

Comment on both amplitude and phase for the cross spectrum.

```
delay = 100;
y1b = [randn(1,delay) x1(1:N-delay)];
rxyPxyplot(x1,y1b,'none',fs)
```

Compare amplitude and phase from before

Examine the characteristics

$$P_{xy}(-\omega) = P_{xy}^{*}(\omega)$$

$$P_{xy}(-\omega) = P_{yx}(\omega)$$

```
rxyPxyplot(x1,y1b,'none',fs)
rxyPxyplot(y1b,x1,'none',fs)
```

y1 is set to a low pass filtrated version of x1

```
h = fir1(50, 1000/(fs/2));
y1c = filter(h,1,x1);
rxyPxyplot(x1,y1c,'none',fs)
```

use zoom-function

How can the delay of the filter be demonstrated?

Example 2: x2 = sinus-signal, y2 = x2 + white noise

```
f1 = 400;
x2 = sin(2*pi*f1*n*Ts);
y2 = x2 + randn(1,N);
rxyPxyplot(x2,y2,'none',fs)
```

Use zoom-function

Calculation of cross spectrum in two ways

As previously examined, the connection is

$$r_{xy}(k) = x(k) * y(-k)$$

In the frequency domain this corresponds to

$$P_{xy}(\omega) = X(\omega) \cdot Y^{*}(\omega)$$

Try, if necessary, to prove this by means of one of the examples shown above.

5. Possibilities for further experiments:
The theoretical foundation for the correlation functions

When calculating and estimating the correlation functions, then – according to the mathematical definition – you shall cancel out by averaging over the ensamble or the discharge space of possible signals, i.e. all the possible signal sections in the observed space of time. At practical calculation/estimation this is not possible, instead a time-averaging of one individual member of the ensamle is performed, which is a concrete section of the signal.

Below you will find a possibility of experimenting with this:

1. Consider the stochastic signal

$$x(n) = \cos(0.1 * n + \theta)$$

where θ is equally distributed (rand-function) between 0 and 2*pi.

2. Plot 3 realizations of the signal.

3. Make a matrix x(m,n), where the m-row is a realization of the process and the n-column are the values of the various realizations at the time n, as we now let m rum from 1:100 and n also runs from 1:100.

4. Calculate the ensamble correlation for x(n) by means of calculating

$$r_{xx}(k) = E[x(1)x(1+k)] \qquad (column - show)$$

Plot this. As expected?

5. Now demonstrate that the signal is ergodic (time-averaging = ensamble-averaging) by means of calculating the average time value and time autocorrelation for a certain realization of the signal, i.e. the first row in the matrice.
Compare this with the emsamble values.
Conclusion?

Try suggesting a non-ergodic signal.
Relevant example: the RMS-value of the thermal interference voltage over a resistance depends on the resistance value. If temperature and other outer factors are constant in time, the interference voltage over the resistance be a *stationary* signal. But not ergodic. Because as there will be tolerance in the resistance value (e.g. 1% resistances), the same result will not be reached after averaging out the interference from more resistances as the result of averaging out over time of the interference from one certain resistance.

The prerequisites for the common definition and application of the correlation functions is:

1) the signals are ergodic
2) the signals are WSS, Wide Sense Stationary, i.e. the average values are not changing over time and the correlation functions are only depending on the time distance between the observed signal-samples, not the absolute points of time.

These are the theoretical prerequisites.
Luckily most practical signals are ergodic and if not absolutely stationary, then WSS.